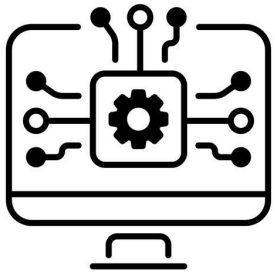
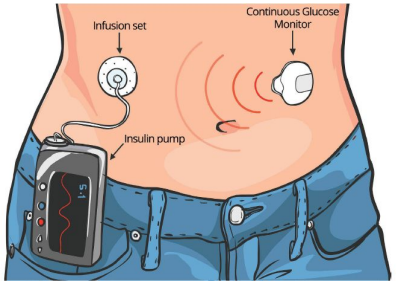
Power and Heat Management in Embedded Systems

By George Elassal

Quick Recap, What are Embedded Systems

- Specialized Computers, Non-General Purpose
- Examples:
  Fitbits, Glucose Monitors, Autopilot Systems …
- Built into larger mechanical Devices
- Processor - Memory - I/O Interfaces
- Deals with Real Time Computing





Why Are Power/Heat Major Design Challenges

- Power/Heat Efficiency Usually comes with Performance Tradeoffs
- Batteries Require Efficiency
- Strict Constraints on Power/Cost/Size/Heat
- Often in Intense Temperature Environments

Motivations and Direction

- Search for General Techniques to Reduce Power/Heat
- Discover How Power and Heat Are Measured
- Ask what tradeoffs are acceptable?
- Ask how are these variables measured?
- What are Researchers looking into now?

Dynamic Voltage and Frequency Scaling

(DVFS)

Survey of Techniques in Power/Heat Management

Dynamic Power Management

Definition: Shutting/Slowing Down Idle or Unused Components [1]

- Places Components into **Different States Dynamically**
- Power Conservation and System **Performance based on the Policy** which decides when and which components are put to sleep.
- Policy Based on System History, Work Load, and Performance Constraints [1]
- Simple Policy = Turn Off Component after Inactivity

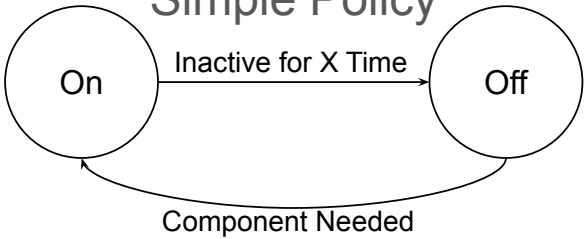
Pros

- Proven to have large improvements on Power Conservation [1]

Cons

- Implementation becomes harder with increases in Hardware Complexity [1]

Simple Policy



Low Power Bus Encoding [1]

Problem: Memory Busses Dissipates Large Amounts of Energy

Solution: Reduce Switching Activity/Hamming Distance

Encoding Methods:

- **Algebraic:** Uses arithmetic or bit-level operations (e.g., Bus-Invert).
- **Permutation:** Reorders data words (e.g., Gray, T0, Pyramid).
- **Probabilistic:** Uses data probability distributions (e.g., Transition or Level Signaling).

Encoding Technique Examples

Gray Code:

- Only one bit changes between consecutive addresses (used in sequential access).
- Example: Counting in binary 0111 → 1000 (4 bits flip)
  vs. Gray code 0100 → 1100 (only 1 bit flips).

Bus-Invert Encoding:

- Before sending a new data word, compare it to the previous one.
- If more than half the bits would change, **invert** all bits and send an “invert” flag.
- This ensures no more than *N/2* bits change at a time.

T0 (Transition) Encoding:

- For sequential address patterns, keeps the bus lines constant and sends only a control bit to indicate an increment — reducing transitions drastically.

Probabilistic Encoding:

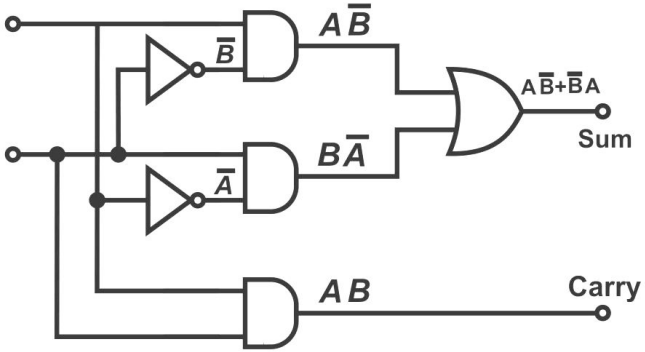
- Uses knowledge of which values occur more often and assigns shorter or less-changing codes to them.

Low Power System Synthesis

- Some Functions run better on Hardware than Software

- Software
  - Function is performed by the processor

- Hardware
  - Function is performed by a dedicated circuit



Dynamic Voltage and Frequency Scaling

$$P_{total} = P_{dynamic} + P_{static}$$

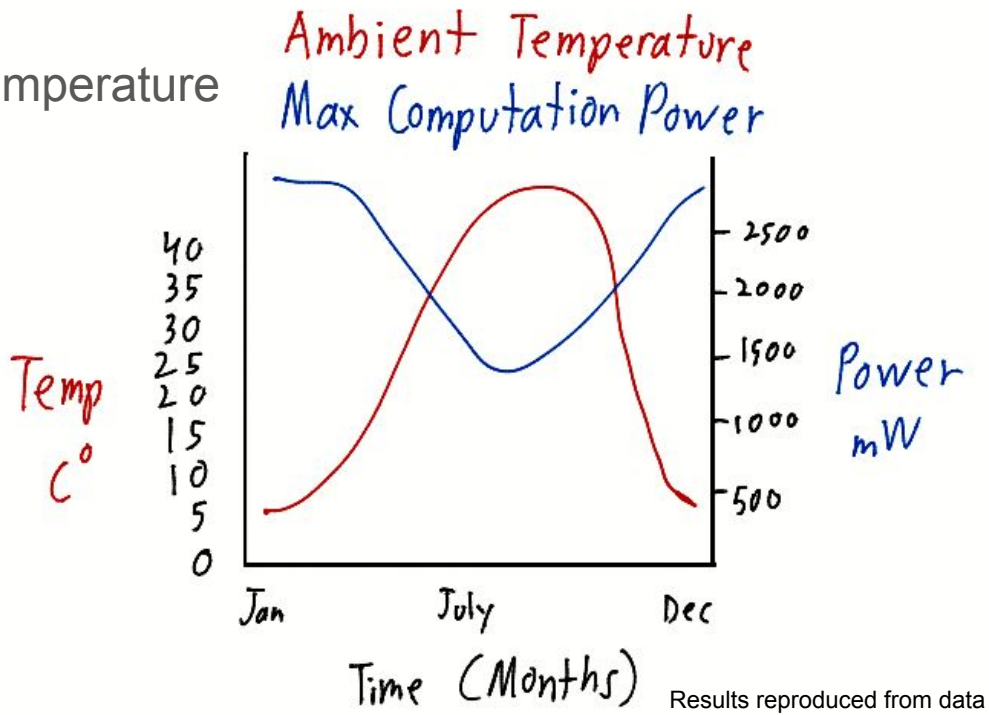
$$P_{dynamic} = \alpha \times C_L \times V^2 \times f$$

$$\text{Execution Time} = (\text{Instructions} \times \text{CPI}) / \text{Frequency}$$

Real Time Thermal Aware Resource Management Framework

Two Main Heat Factors

- Dynamically Changing Ambient Temperature
- Task-Level Power Dissipation



Results reproduced from data in Y. Lee et al., 2018 [2].

Defining Parameters for Voltage - Frequency - Period

Basis of Feasibility Conditions is that Steady Temperature of Task Set is Independent of its Schedule and that a Task Set will eventually reach its Steady Temperature [2]

$$T(\infty, \tau) \leq T_{\max}.$$

$$\downarrow$$

$$\mathrm{PD}(\tau) \leq \mathrm{PB}(\mathrm{T_{amb}})$$

$$\mathrm{PD}(\tau) = \sum_{\tau_i} P_i \cdot \frac{e_i(f)}{p_i} = V^2 \cdot f \cdot \sum_{\tau_i} \alpha_i \cdot \frac{e_i(f)}{p_i}$$

$$\mathrm{PB}(\mathrm{T_{amb}}) = \frac{T_{\max} - T_{\mathrm{amb}}}{R} - V \cdot (\beta_1 \cdot T_{\max} + \beta_0)$$

$$\sum_{\tau_i} \frac{e_i(f)}{p_i} \leq 1. \longrightarrow \sum_{\tau_i} \frac{e_i(f) + I_i^{\min}(T_{\mathrm{amb}})}{p_i} \leq 1$$

Equations reproduced from Y. Lee et al., (2018) [2]

Optimization Problem

$$\underset{f,p_i}{\text{maximize}} \sum_{\tau_i} w_i \cdot \frac{1}{p_i}$$

$$\text{s.t.} \quad \overline{\text{C1}}\text{: PD}(\tau) = V^2 \cdot f \cdot \sum_{\tau_i} \alpha_i \cdot \frac{e_i\left(f\right)}{p_i} \leq \text{PB}(\text{T}_{\text{amb}})$$

$$\overline{\text{C2}}\text{: } \sum_{\tau_i} \frac{e_i\left(f\right) + I_i^{\min}\left(T_{\text{amb}}\right)}{p_i} \leq 1$$

$$f \in \left[f_{\min}, \ldots, f_{\max}\right].$$

$$\forall \tau_i \quad p_i^{\min} \leq p_i \leq p_i^{\max}.$$

Equations reproduced from
Y. Lee et al., (2018) [2]

Online Idle Time Scheduling

- Before each task can run the system must be at a safe temperature first

$$T_i^{\text{safe}}\left(e_i\left(f\right), T_{\text{amb}}\right) = T_i^{\infty}\left(T_{\text{amb}}\right) - \frac{T_i^{\infty}\left(T_{\text{amb}}\right) - T_{\max}}{e^{-\frac{e_i(f)}{R \cdot C}}}$$

- Calculate Idle Time necessary to reach safe temperature

$$t_{\text{idle}}\left(e_i\left(f\right), T_{\text{amb}}\right) = R \cdot C \cdot ln\left(\frac{T_{\max} - T_0^{\infty}\left(T_{\text{amb}}\right)}{T_i^{\text{safe}}\left(e_i\left(f\right), T_{\text{amb}}\right) - T_0^{\infty}\left(T_{\text{amb}}\right)}\right)$$

- Allow Pre-emption, Find Minimum Idle Time Required for Each Task

$$\frac{\partial}{\partial m_i}\left(m_i \cdot t_{idle}\left(\frac{e_i(f)}{m_i}, T_{amb}\right)\right) \longrightarrow I_i^{\min}\left(T_{\text{amb}}\right) = m_i^{\max} \cdot t_{\text{idle}}\left(\frac{e_i\left(f\right)}{m_i^{\max}}, T_{\text{amb}}\right)$$

$$\sum_{\tau_i} \frac{e_i\left(f\right) + I_i^{\min}\left(T_{\text{amb}}\right)}{p_i} \leq 1$$

Equations reproduced from Y. Lee et al., (2018) [2]

Slack Dispersal

- Static Idle Time Allocation is In-Efficient
- Update Slack/Allocated Idle Time When:
  - Job Finishes
  - Job Ends
  - DVFS Parameters Change

$$I_i(t_{cur}) = I_i^{min}(T_{amb}) + S(t_{cur}, d_1(t_{cur})) \cdot \frac{P_i \cdot \frac{e_i(f)}{p_i}}{\sum_{\tau_i} P_i \cdot \frac{e_i(f)}{p_i}}$$

Equations reproduced from
Y. Lee et al., (2018) [2]

Results

- EDF 1 - Full Power
  - Overheats 77% of the time
- EDF 2 - 60% Frequency
  - 63% task rate
- RT-TMC - Similar to RT-TRM but without Thermal Aware Scheduling
  - 3% violations and 67% task rate
- RT-TRM
  - No Violations and 80% task rate [2]

Power-Aware Design Techniques of Secure Multimode Embedded Systems

K.Jiang et al., (2016) [3]

- Maximize:
  - Quality of Service
- Reach Required Levels of:
  - Confidentiality
  - Intrusion Detection Accuracy
- Minimize :
  - Power Consumption
  - Heat Generation

Modes and Pareto Spaces

- Each Mode (a set of tasks) has many different possible configurations
- Configurations can vary in:
  - Voltage & Frequency (DVFS)
  - Encryption Strength (multiple cipher variants)

    Intrusion Detection Rate
  - Optional QoS execution time
- **Not Enough Memory** to Store Pareto Spaces for each Mode
- **Offline** Pareto Space **Calculation and Filtering**

Offline vs Runtime

1. Check if Current Mode has Pareto Space saved
   - Yes :
     - Match Configuration to Pareto Space
   - No
     - Match Configuration to Nearby Supermode
     - Trim Unused Tasks and Reallocate Resources
     - Improve QoS

Conclusions

Offline Phase (Design-Time)

- They tested systems with **4, 6, 8, and 10 tasks**
- Storing *more* Pareto spaces → **higher performance**, but longer preprocessing time
- Even storing **only ~30% of modes** gave **over 100% improvement** vs. storing only the mandatory modes [3]

Runtime Phase

- Selecting a configuration is **fast**:
  - **~0.1 ms** when the Pareto space is available

    Up to **~0.7 ms** when deriving from a supermode [3]
- This overhead is **negligible** relative to task periods (microseconds to milliseconds)

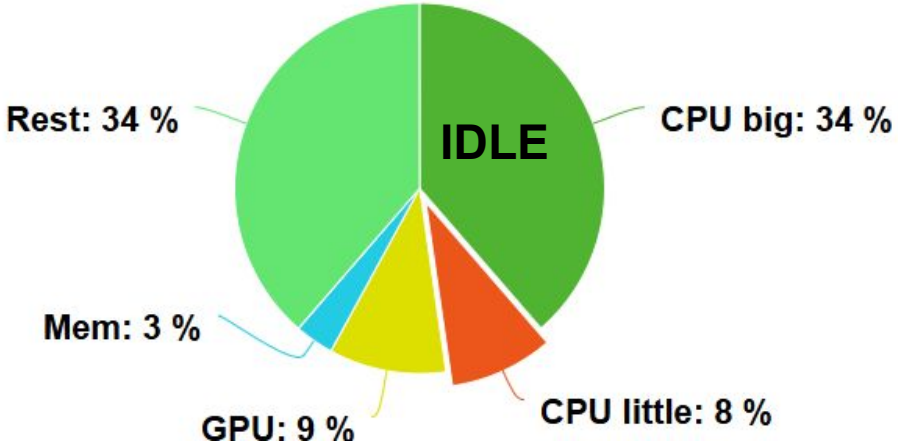
Improvements

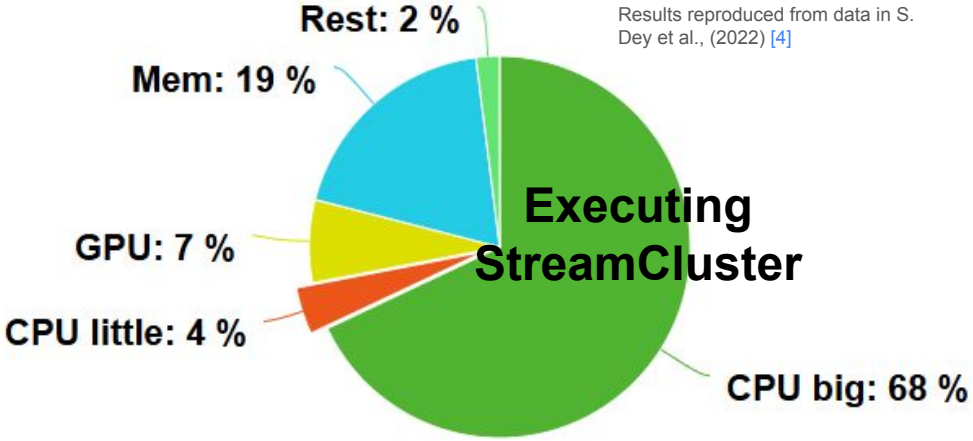
- Improved quality of service by 40–80%[3]

CPU-GPU-Memory DVFS (Especially in Mobile Devices)

S. Dey et al., (2022) [4]

- Most DVFS systems act only on a single CPU or CPU + GPU
- Modern Embedded Systems are Heterogeneous
  - Big CPUs
  - Little Cpus
  - GPUs
  - Memory
- Memory Can use about 20% of the Total Power





Results reproduced from data in S. Dey et al., (2022) [4]

CGM-DVFS

1. **Profiling (Offline / Background)**
   - Try combinations of:
     - Big CPU freq
       Little CPU freq
     - GPU freq
     - Memory freq
   - Record for each combination:
     - Performance (FPS or execution time)
     - Power consumption
     - Temperature (thermal impact)
2. **Design Space Reduction**
   - They don't test *every* setting → they cluster DVFS settings that give similar performance.
   - This reduces the search space from **thousands** → **216 meaningful configurations.** [4]
3. **Runtime Selection**
   - The system checks what performance level is required (maintain 60 FPS).
   - Chooses **Lowest Power** Profiled Configuration that meets performance requirements
   - Then it **sets the DVFS levels** accordingly.

Results and Conclusion

- Tested on Odroid XU4 (Exynos 5422 MPSoC) across real workloads (YOLO, Face Detection, Video Streaming, FFT).
- **~26–33% power reduction** vs default governors & prior DVFS techniques.
- **~21–26% lower peak CPU temperature**.
- Runtime overhead for DVFS decision is **~30 ms** — very lightweight. [4]

Conclusions and Future Directions

- Multiple Factors Affect Power/Heat
  - Ambient Temperature
  - Security Tasks
  - Schedules
  - Microarchitecture Design
- Thermal Modeling is the base of DVFS
- Modern Systems are Heterogeneous!
- Real Time vs Offline Profiling/Optimization vs ML/AI prediction

Bibliography

[1] K. Jiang, Y. Zhang, Q. Zhu, and M. A. Al Faruque, “Power-Aware Design Techniques of Secure Multimode Embedded Systems,” *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 21, no. 3, Feb. 2016, Art. no. 45, doi: 10.1145/2801152.

[2] Y. Lee, H. S. Chwa, K. G. Shin, and S. Wang, “Thermal-Aware Resource Management for Embedded Real-Time Systems,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 11, pp. 2857–2868, Nov. 2018, doi: 10.1109/TCAD.2018.2857279.

[3] M. Pedram, “Power and Heat Management in Embedded Systems,” in *Proceedings of the 2001 International Symposium on Low Power Electronics and Design (ISLPED ’01)*, Aug. 2001, pp. 1–5, doi: 10.1145/370155.370333.

[4] S. Dey, S. Isuwa, S. Saha, A. K. Singh, and K. McDonald-Maier, “CPU-GPU-Memory DVFS for Power-Efficient MPSoC in Mobile Cyber Physical Systems,” *Future Internet*, vol. 14, no. 3, Art. no. 91, Mar. 2022, doi:10.3390/fi14030091.